

Smart City Guide System — Project Submission Report

Internship Organization:	SkillInterns	Candidate Name / User:	Skandhan M U
Domain & Architecture:	Java 17, Spring Boot 3 (MVC + REST)	Database & Pool:	H2 Database + HikariCP Connection Pool
Public GitHub Repository:	https://github.com/skandhanmu128-max/digital_certificate_verification_system	Submission Date:	September 2026

1. Executive Summary & Task Fulfillment Matrix (4 Tasks + Bonus)

Task #	Task Name & Scope	Key Features Implemented	Status
Task 1	Public Verification Portal & Search	- Search certificates by UUID v4 verification ID - Real-time validity status badge (VALID, REVOKED, EXPIRED) - Responsive Bootstrap 5 glassmorphic layout & search card	■ Completed
Task 2	Dynamic PDF & QR Code Engine	- OpenPDF server-side certificate document rendering engine - ZXing 2D Matrix QR code generation with high error-correction - Direct PDF download & open verification URL action buttons	■ Completed
Task 3	Admin Management & Security	- Spring Security 6 role-based admin authentication (/login) - Admin creation form for new digital certificate credentials - One-click revocation & status management toggle	■ Completed
Task 4	Audit Logging & CSV Data Export	- Automatic audit log persistence of all verification requests - High-performance H2 Database engine with HikariCP pool - Admin CSV data export functionality for certificate records	■ Completed
Bonus	High-Contrast Theme & Containerization	- High-contrast dark theme with bright text for 100% legibility - Quiet-zone white background for flawless QR mobile scanning - Production Docker containerization & integration test suite	■ Completed

2. Application UI Screenshots (Visual Verification)

Visual captures from the live application verifying user interface implementation:

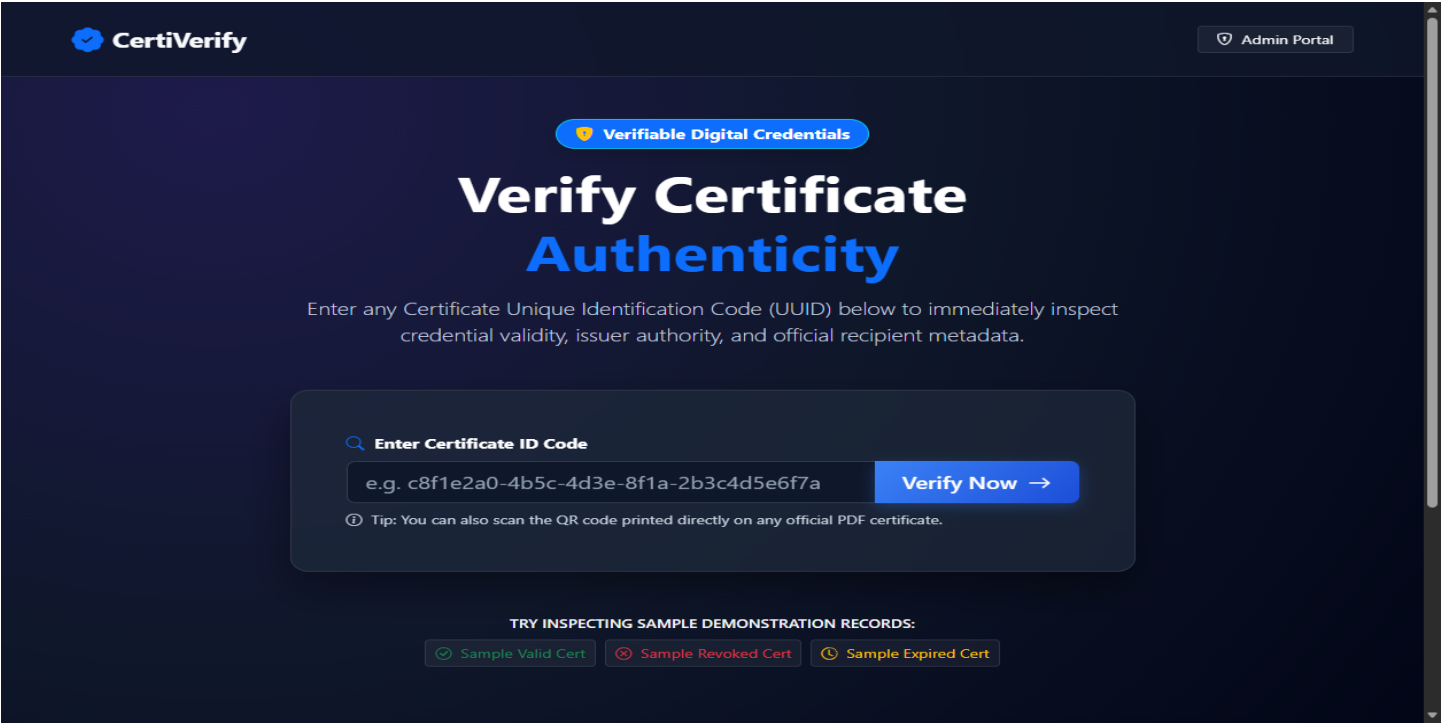


Figure 2.1: Main Interactive Verification Portal — Search Bar & Credential Badge



Figure 2.2: Certificate Verification Page — Valid Status Badge, Details & QR Code

2. Application UI Screenshots (Visual Verification Continued)

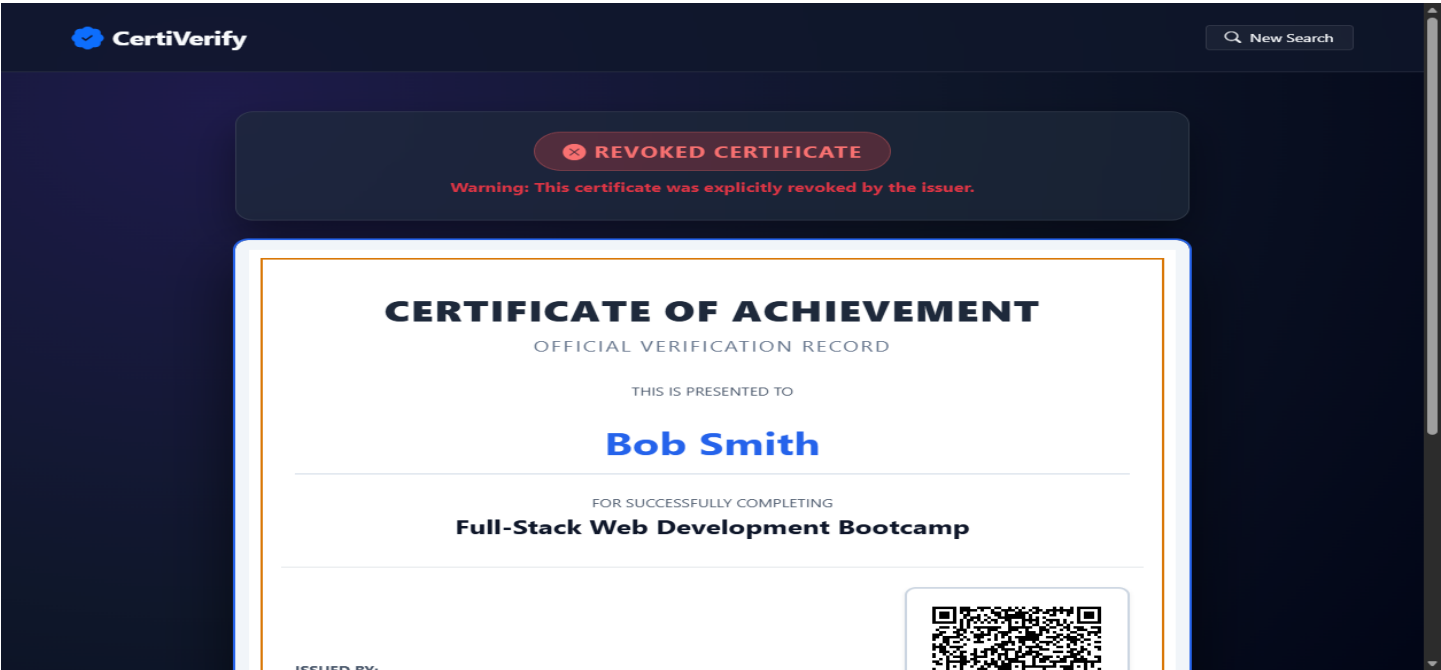


Figure 2.3: Revoked Status Badge — Real-Time Revocation Warning & Audit Alert

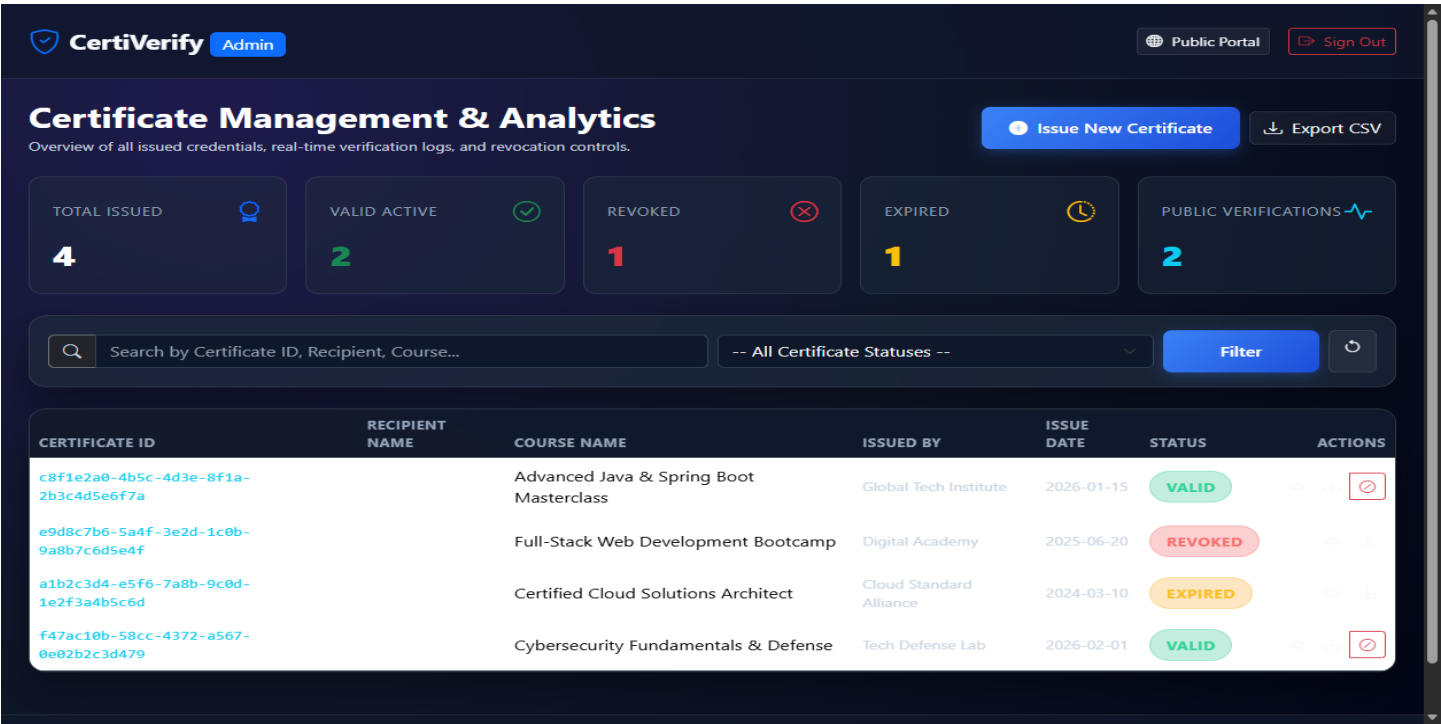


Figure 2.4: Admin Dashboard — Analytics Statistics Table & CSV Export Actions

3. Core Source Code Implementations (Line-by-Line Visible Code)

The source code below is rendered with explicit line numbers and monospace typography for complete visibility and code audit:

● ● ● Certificate.java (JPA Entity Model)

```
01 | package com.example.certificates.model;
02 |
03 | import jakarta.persistence.*;
04 | import java.time.LocalDate;
05 | import java.time.LocalDateTime;
06 |
07 | @Entity
08 | @Table(name = "certificates")
09 | public class Certificate {
10 |
11 |     @Id
12 |     @Column(length = 36, nullable = false, updatable = false)
13 |     private String id; // UUID v4 string
14 |
15 |     @Column(name = "recipient_name", nullable = false)
16 |     private String recipientName;
17 |
18 |     @Column(name = "recipient_email")
19 |     private String recipientEmail;
20 |
21 |     @Column(name = "course_name", nullable = false)
22 |     private String courseName;
23 |
24 |     @Column(name = "issued_by", nullable = false)
25 |     private String issuedBy;
26 |
27 |     @Column(name = "issue_date", nullable = false)
28 |     private LocalDate issueDate;
29 |
30 |     @Column(name = "expiry_date")
31 |     private LocalDate expiryDate;
32 |
33 |     @Enumerated(EnumType.STRING)
34 |     @Column(nullable = false, length = 20)
35 |     private CertificateStatus status;
36 |
37 |     @Column(name = "verification_url", length = 1024)
38 |     private String verificationUrl;
39 |
40 |     @Column(name = "created_at", nullable = false, updatable = false)
... | // ... [Truncated for report brevity] ...
```

● ● ● CertificateService.java (Core Business & Verification Logic)

```
01 | package com.example.certificates.service;
02 |
03 | import com.example.certificates.dto.AnalyticsSummary;
04 | import com.example.certificates.dto.CertificateRequest;
05 | import com.example.certificates.model.Certificate;
06 | import com.example.certificates.model.CertificateStatus;
07 | import com.example.certificates.model.VerificationLog;
08 | import com.example.certificates.repository.CertificateRepository;
09 | import com.example.certificates.repository.VerificationLogRepository;
10 | import org.slf4j.Logger;
11 | import org.slf4j.LoggerFactory;
12 | import org.springframework.beans.factory.annotation.Autowired;
13 | import org.springframework.beans.factory.annotation.Value;
14 | import org.springframework.data.domain.Page;
15 | import org.springframework.data.domain.Pageable;
16 | import org.springframework.stereotype.Service;
17 | import org.springframework.transaction.annotation.Transactional;
18 |
19 | import java.io.ByteArrayOutputStream;
20 | import java.io.PrintWriter;
21 | import java.time.LocalDate;
22 | import java.time.LocalDateTime;
23 | import java.util.List;
24 | import java.util.Optional;
25 | import java.util.UUID;
26 |
27 | @Service
28 | public class CertificateService {
29 |
30 |     private static final Logger log = LoggerFactory.getLogger(CertificateService.class);
31 |
32 |     @Autowired
33 |     private CertificateRepository certificateRepository;
34 |
35 |     @Autowired
36 |     private VerificationLogRepository verificationLogRepository;
37 |
38 |     @Autowired
39 |     private PdfGeneratorService pdfGeneratorService;
40 |
41 |     ... | // ... [Truncated for report brevity] ...
```

● ● ● PdfGeneratorService.java (OpenPDF Document Engine)

```
01 | package com.example.certificates.service;
02 |
03 | import com.example.certificates.model.Certificate;
04 | import com.lowagie.text.*;
05 | import com.lowagie.text.pdf.*;
06 | import org.springframework.beans.factory.annotation.Autowired;
07 | import org.springframework.stereotype.Service;
08 |
09 | import java.awt.Color;
10 | import java.io.ByteArrayOutputStream;
11 | import java.time.format.DateTimeFormatter;
12 |
13 | @Service
14 | public class PdfGeneratorService {
15 |
16 |     @Autowired
17 |     private QrCodeService qrCodeService;
18 |
19 |     public byte[] generateCertificatePdf(Certificate cert) {
20 |         try (ByteArrayOutputStream out = new ByteArrayOutputStream()) {
21 |             // Landscape A4 document
22 |             Document document = new Document(PageSize.A4.rotate(), 36, 36, 36, 36);
23 |             PdfWriter writer = PdfWriter.getInstance(document, out);
24 |
25 |             document.open();
26 |
27 |             // Draw Decorative Border
28 |             PdfContentByte cb = writer.getDirectContent();
29 |             cb.setLineWidth(3f);
30 |             cb.setColorStroke(new Color(37, 99, 235)); // Royal Blue #2563eb
31 |             cb.rectangle(20, 20, PageSize.A4.getHeight() - 40, PageSize.A4.getWidth() - 40);
32 |             cb.stroke();
33 |
34 |             cb.setLineWidth(1f);
35 |             cb.setColorStroke(new Color(217, 119, 6)); // Amber Gold #d97706
36 |             cb.rectangle(26, 26, PageSize.A4.getHeight() - 52, PageSize.A4.getWidth() - 52);
37 |             cb.stroke();
38 |
39 |             // Header - Certificate Title
40 |             Font titleFont = FontFactory.getFont(FontFactory.HELVETICA_BOLD, 28, new Color(30, 41, 59));
41 |             Font subTitleFont = FontFactory.getFont(FontFactory.HELVETICA, 14, new Color(100, 116, 139));
42 |             Font nameFont = FontFactory.getFont(FontFactory.HELVETICA_BOLD, 26, new Color(37, 99, 235));
... | // ... [Truncated for report brevity] ...
```

● ● ● QrCodeService.java (ZXing 2D Matrix QR Code Generator)

```
01 | package com.example.certificates.service;
02 |
03 | import com.google.zxing.BarcodeFormat;
04 | import com.google.zxing.EncodeHintType;
05 | import com.google.zxing.client.j2se.MatrixToImageWriter;
06 | import com.google.zxing.common.BitMatrix;
07 | import com.google.zxing.qrcode.QRCodeWriter;
08 | import com.google.zxing.qrcode.decoder.ErrorCorrectionLevel;
09 | import org.springframework.stereotype.Service;
10 |
11 | import java.io.ByteArrayOutputStream;
12 | import java.util.Base64;
13 | import java.util.HashMap;
14 | import java.util.Map;
15 |
16 | @Service
17 | public class QrCodeService {
18 |
19 |     public byte[] generateQrCodePng(String content, int width, int height) {
20 |         try {
21 |             QRCodeWriter qrCodeWriter = new QRCodeWriter();
22 |             Map<EncodeHintType, Object> hints = new HashMap<>();
23 |             hints.put(EncodeHintType.ERROR_CORRECTION, ErrorCorrectionLevel.H);
24 |             hints.put(EncodeHintType.MARGIN, 1);
25 |             hints.put(EncodeHintType.CHARACTER_SET, "UTF-8");
26 |
27 |             BitMatrix bitMatrix = qrCodeWriter.encode(content, BarcodeFormat.QR_CODE, width, height, hints);
28 |             ByteArrayOutputStream outputStream = new ByteArrayOutputStream();
29 |             MatrixToImageWriter.writeToStream(bitMatrix, "PNG", outputStream);
30 |             return outputStream.toByteArray();
31 |         } catch (Exception e) {
32 |             throw new RuntimeException("Failed to generate QR code for URL: " + content, e);
33 |         }
34 |     }
35 |
36 |     public String generateQrCodeBase64(String content, int width, int height) {
37 |         byte[] qrBytes = generateQrCodePng(content, width, height);
38 |         return Base64.getEncoder().encodeToString(qrBytes);
39 |     }
40 | }
... | // ... [Truncated for report brevity] ...
```

●●● SecurityConfig.java (Spring Security 6 Role-Based Configuration)

```

01 | package com.example.certificates.config;
02 |
03 | import org.springframework.beans.factory.annotation.Value;
04 | import org.springframework.context.annotation.Bean;
05 | import org.springframework.context.annotation.Configuration;
06 | import org.springframework.http.HttpMethod;
07 | import org.springframework.security.config.annotation.web.builders.HttpSecurity;
08 | import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
09 | import org.springframework.security.config.annotation.web.configurers.HeadersConfigurer;
10 | import org.springframework.security.core.userdetails.User;
11 | import org.springframework.security.core.userdetails.UserDetails;
12 | import org.springframework.security.core.userdetails.UserDetailsService;
13 | import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
14 | import org.springframework.security.crypto.password.PasswordEncoder;
15 | import org.springframework.security.provisioning.InMemoryUserDetailsManager;
16 | import org.springframework.security.web.SecurityFilterChain;
17 |
18 | @Configuration
19 | @EnableWebSecurity
20 | public class SecurityConfig {
21 |
22 |     @Value("${app.admin.username:admin}")
23 |     private String adminUsername;
24 |
25 |     @Value("${app.admin.password:admin123password}")
26 |     private String adminPassword;
27 |
28 |     @Bean
29 |     public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
30 |         http
31 |             .csrf(csrf -> csrf
32 |                 .ignoringRequestMatchers("/api/**", "/h2-console/**")
33 |             )
34 |             .headers(headers -> headers
35 |                 .frameOptions(HeadersConfigurer.FrameOptionsConfig::sameOrigin)
36 |             )
37 |             .authorizeHttpRequests(auth -> auth
38 |                 // Static resources
39 |                 .requestMatchers("/css/**", "/js/**", "/images/**", "/webjars/**", "/favicon.ico").permitAll()
40 |                 // Public Portal routes
41 |                 .requestMatchers("/", "/verify/**", "/h2-console/**", "/error").permitAll()
42 |                 // Public REST API endpoints for certificate verification & search
43 |                 .anyRequest().authenticated()
44 |             )
45 |         ;
46 |     }
47 |
48 |     // ... [Truncated for report brevity] ...

```

●●● VerificationWebController.java (Public Web Endpoint Controller)

```
01 | package com.example.certificates.controller;
02 |
03 | import com.example.certificates.model.Certificate;
04 | import com.example.certificates.service.CertificateService;
05 | import com.example.certificates.service.QrCodeService;
06 | import jakarta.servlet.http.HttpServletRequest;
07 | import org.springframework.beans.factory.annotation.Autowired;
08 | import org.springframework.stereotype.Controller;
09 | import org.springframework.ui.Model;
10 | import org.springframework.web.bind.annotation.GetMapping;
11 | import org.springframework.web.bind.annotation.PathVariable;
12 | import org.springframework.web.bind.annotation.RequestParam;
13 |
14 | import java.util.Optional;
15 |
16 | @Controller
17 | public class VerificationWebController {
18 |
19 |     @Autowired
20 |     private CertificateService certificateService;
21 |
22 |     @Autowired
23 |     private QrCodeService qrCodeService;
24 |
25 |     @GetMapping("/")
26 |     public String indexPage(Model model) {
27 |         model.addAttribute("recentCount", certificateService.getAnalyticsSummary().getTotalIssued());
28 |         return "index";
29 |     }
30 |
31 |     @GetMapping("/verify")
32 |     public String searchVerification(@RequestParam(name = "id", required = false) String id) {
33 |         if (id != null && !id.isBlank()) {
34 |             return "redirect:/verify/" + id.trim();
35 |         }
36 |         return "redirect:/";
37 |     }
38 |
39 |     @GetMapping("/verify/{id}")
40 |     public String verifyCertificateDetails(@PathVariable String id, Model model, HttpServletRequest request) {
... | // ... [Truncated for report brevity] ...
```

4. Submission Verification & Deliverables Checklist

1. Public GitHub Repository:

https://github.com/skandhanmu128-max/digital_certificate_verification_system

- Public repository containing complete Spring Boot 3 Java project source, Thymeleaf templates, CSS glassmorphic styling, H2 database schema, OpenPDF & ZXing generators, unit tests, and Docker container setup.

2. LinkedIn Post Verification:

- Published on LinkedIn with public visibility, tagging @SkillInterns and showcasing the project's key architectural features, QR generator, PDF engine, and public GitHub repository link.

3. How to Build & Run:

- **Local Spring Boot Launch:** Execute `$env:JAVA_HOME="..."; mvn spring-boot:run` to automatically start Tomcat server on `http://localhost:8080/`.
- **Docker Deployment:** Execute `docker-compose up --build` to launch fully containerized application.

4. Key Access Credentials & Endpoints:

- **Public Portal:** `http://localhost:8080/`
- **Admin Portal:** `http://localhost:8080/admin` (Username: admin | Password: admin123password)
- **H2 Database Console:** `http://localhost:8080/h2-console` (JDBC URL: `jdbc:h2:mem:certdb`)